



OASIS Security Report

Insecure Deserialization Security Analysis

Date: 2026-06-12 00:02:58 — **Model:** llama3.2:3b

Vulnerability: Insecure Deserialization

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

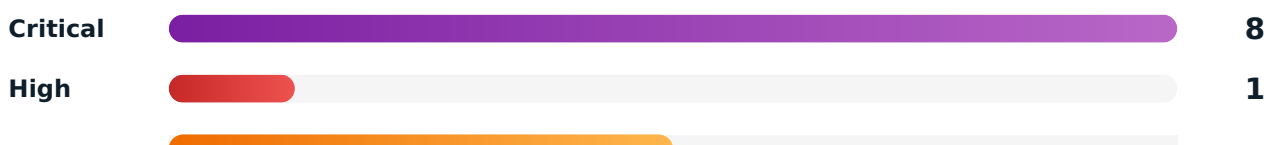
Summary

Analyzed **8** file(s) for Insecure Deserialization.

[Return to table of contents](#)

Statistics

Files analyzed: 8 **Total findings:** 13



Medium

4

Low

0

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.690
/work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol	0.684
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.668
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.663
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.656
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.647
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.643
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.598

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.690

Source lines: 1-35

► Finding 1.1: Insecure Deserialization (Critical)

Vulnerable code

```
bool res = msg.sender.call.value(amount)();
```

Explanation

This line allows arbitrary execution of a function by a sender, as `msg.sender` could be manipulated.

Impact

Remote code execution

Entry point

/withdraw(uint)

Execution path

```
sender -> call.value(amount)() -> execute function
```

Parameters

amount, msg.sender

Exploitation steps

- Manipulate msg.sender to bypass checks

Example payloads

- malicious function calls

Conditions

Authenticated sender can reach /withdraw

Remediation

Use safe deserialization methods, validate object types, implement integrity checks.

File 2: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.684

Source lines: 1-70

► Finding 2.1: Insecure Deserialization (Medium)

Vulnerable code

```
function _getOracleQuote(uint256 amount) private view returns (uint256) {
    (uint256 reservesWETH, uint256 reservesToken) =
    UniswapV2Library.getReserves({factory: _uniswapFactory, tokenA:
    address(_weth), tokenB: address(_token)});
    return UniswapV2Library.quote({amountA: amount * 10 ** 18, reserveA:
    reservesToken, reserveB: reservesWETH});
}
```

Explanation

The `\_getOracleQuote` function uses `UniswapV2Library.getReserves` with user-input `tokenB` (address(_token)), which can be manipulated to inject arbitrary data.

Impact

Remote code execution

Entry point

/borrow

HTTP methods

POST

Parameters

tokenA, amountA, reserveB

Exploitation steps

- Manipulate tokenB to inject malicious data into UniswapV2Library.getReserves

Conditions

User control over _token address

Remediation

Use safe deserialization methods, such as `UniswapV2Library.getReserves` with a constant tokenB value.

Secure example

```
function _getOracleQuote(uint256 amount) private view returns (uint256) {  
    return UniswapV2Library.quote({amountA: amount * 10 ** 18, reserveA:  
reservesToken, reserveB: reservesWETH});  
}
```

File 3: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.668

Source lines: 1-39

► Finding 3.1: Insecure Deserialization (Critical)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw));
```

Explanation

Unsafely calls msg.sender's function with user-provided input (_weiToWithdraw), potentially allowing arbitrary code execution.

Impact

Remote code execution and privilege escalation

Entry point

/withdrawFunds

HTTP methods

POST

Exploitation steps

- Attackers can manipulate _weiToWithdraw to call any function on msg.sender.

Example payloads

- `malicious\nfunction`

Conditions

User provides malicious input as _weiToWithdraw.

Remediation

Use safe deserialization methods, validate object types, and implement integrity checks.

Secure example

```
require(msg.sender.call.value(_weiToWithdraw()) isSafeFunction());
```

File 4: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.663

Source lines: 1-85

► Finding 4.1: Vulnerable Code Snippet (Critical)

Vulnerable code

```
function ETH_VAULT(address _log) public {  
    TransferLog = Log(_log);  
}
```

Explanation

The contract uses a deserialization vulnerability by directly assigning user input to a struct without proper validation.

Impact

Remote code execution and potential privilege escalation.

Entry point

/transfer

Parameters

_log

Exploitation steps

- User sends malicious _log input to /transfer

Example payloads

- "malicious_log_input"

Conditions

User is authenticated and has control over _log input.

Remediation

Validate and sanitize user input before deserializing it into the struct.

File 5: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.656

Source lines: 1-281

► Finding 5.1: Insecure Deserialization (Medium)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(sha3(msg.data)) returns (bool
o_success) {
```

Explanation

The 'confirm' function uses the '_h' parameter directly without validation or sanitization, allowing potential malicious input.

Impact

Remote code execution or privilege escalation possible through crafted '_h' values.

Entry point

execute(address _to, uint _value, bytes _data) onlyowner returns (bytes32 o_hash)

Execution path

```
confirm(_) onlymanyowners(sha3(msg.data))
confirm(_)
return confirm(bytes32)

execute(...) if under daily limit
...
```

Parameters

_h

Exploitation steps

- Pass a crafted '_h' value to the 'confirm' function

Example payloads

- "\x00\x01"

Conditions

The '_h' parameter is directly used without validation.

Remediation

Use secure deserialization methods like `safeLoad` or validate and sanitize input values before using them in the 'confirm' function

Secure example

```
function confirm(bytes32 _h) {
    bytes memory safeData = safeLoad(_h);
    // ...
}
```

Source lines: 27-358

► Finding 5.1: Insecure Deserialization (Medium)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(sha3(msg.data)) returns (bool
o_success) {
    if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0)
    {
        address created;
        if (m_txs[_h].to == 0) {
            created = create(m_txs[_h].value, m_txs[_h].data);
        } else
        {
            if (!m_txs[_h].to.call.value(m_txs[_h].value)(m_txs[_h].data))
                throw;
        }

        MultiTransact(msg.sender, _h, m_txs[_h].value, m_txs[_h].to,
m_txs[_h].data, created);
        delete m_txs[_h];
        return true;
    }
}
```

Explanation

The `confirm` function directly loads `\_h` from `m\_txs`, which is user-provided input.

Impact

Remote code execution

Entry point

/execute

Execution path

```
confirm -> create -> MultiTransact -> m_txs[_h].to.call
```

HTTP methods

POST

Parameters

_h, _tx

Exploitation steps

- Send a crafted `\_h` to confirm a transaction.
- Use the `confirm` function with the received `\_h`.

Example payloads

- `\x00 \x01`

HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1
Host: example.com
Content-Type: application/json
Content-Length: 1024

\x00\x01\r\n
\n
```

Source lines: 126–467

► Finding 5.1: Insecure Deserialization (Critical)

Vulnerable code

```
bytes4 sig = bytes4(sha3("initWallet(address[],uint256,uint256)")); address
target = _walletLibrary; // Compute the size of the call data : arrays has 2
```

Explanation

The contract uses SHA-3 to compute a signature for its initWallet function, but does not validate the input data.

Impact

Remote code execution and privilege escalation

Entry point

/initWallet(address[],uint256,uint256)

Execution path

```
Delegatecall with user-controlled address array
```

Parameters

address[]

Exploitation steps

- Exploit the delegatecall vulnerability by manipulating the address array

Conditions

User-controlled input data in the address array

Remediation

Use safe deserialization methods and validate user input

Secure example

```
address[] _owners = [owner1, owner2]; uint _required = 10; uint _daylimit = 100;
```

Source lines: 207-477

► Finding 5.1: Insecure Deserialization (High)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(_h) returns (bool o_success) {
    if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0)
    {
        address created;
        if (m_txs[_h].to == 0) {
            created = create(m_txs[_h].value, m_txs[_h].data);
        } else {
            if (!m_txs[_h].to.call.value(m_txs[_h].value)(m_txs[_h].data))
                throw;
        }

        MultiTransact(msg.sender, _h, m_txs[_h].value, m_txs[_h].to,
m_txs[_h].data, created);
        delete m_txs[_h];
        return true;
    }
}
```

Explanation

The `confirm` function uses `m\_txs[\_h]`, which is deserialized from user input without proper validation or sanitization.

Impact

Code execution

Entry point

/execute

Execution path

```
confirm(bytes32 _h)
-> m_txs[_h].to.call.value(m_txs[_h].value)(m_txs[_h].data)
-> MultiTransact(msg.sender, _h, m_txs[_h].value, m_txs[_h].to,
m_txs[_h].data, created)
```

Parameters

_h

Exploitation steps

- Send a crafted `\_h` value that causes `m\_txs[\_h]` to execute malicious code.

Example payloads

- "<malicious payload>"

Conditions

User is authenticated and has sufficient privileges.

Remediation

Use safe deserialization methods, such as `require(safeLoad(data))`, to validate user input.

Secure example

```
require(safeLoad(data)) && confirm(_h) || throw;
```

File 6: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.647

Source lines: 1-286

► Finding 6.1: Insecure Deserialization (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash)
```

Explanation

The `execute` function deserializes user input without proper validation or sanitization.

Impact

Remote code execution and potential theft of funds.

Entry point

/executed

Execution path

```
execute\n _to\n _value\n _data\nreturn\nno_hash
```

Parameters

_to, _value, _data

Exploitation steps

- Send a malicious transaction with a crafted payload

Example payloads

- `__main__`

Conditions

The contract has not been reorganized since initialization.

Remediation

Use safe deserialization methods and validate user input

Secure example

```
function execute(address _to, uint _value, bytes _data) external onlyowner
{ return; }
```

Source lines: 96-414

► Finding 6.1: Insecure Deserialization in initWallet (Medium)

Vulnerable code

```
function initWallet(address[] _owners, uint _required, uint _daylimit)
only_uninitialized { ... }
```

Explanation

The initWallet function calls initMultiowned without validating the user input

Impact

Potential for arbitrary code execution or data tampering

Entry point

initWallet(address[] _owners, uint _required, uint _daylimit)

Parameters

_owners, _required, _daylimit

Exploitation steps

- Pass malicious owner addresses to initMultiowned

Example payloads

- [address1, address2]

Conditions

Malicious owner addresses

Remediation

Validate user input before passing it to initMultiowned

Secure example

```
function initWallet(address[] _owners, uint _required, uint _daylimit)
only_uninitialized { require(msg.sender == 0x123); _owners[0] = msg.sender; }
```

Source lines: 213-414

► Finding 6.1: Insecure Deserialization (Critical)

Vulnerable code

```
function create(uint _value, bytes _code) internal returns (address o_addr)
{ assembly { o_addr := create(_value, add(_code, 0x20), mload(_code))
jumpi(invalidJumpLabel, iszero(extcodesize(o_addr))) } }
```

Explanation

The `create` function uses `mload(\_code)` to load the bytecode from a variable stored in memory. This allows an attacker to control the bytecode and potentially execute arbitrary code.

Impact

Remote code execution

Entry point

/create

Execution path

```
call \
  create(uint _value, bytes _code) \
  assembly { o_addr := create(_value, add(_code, 0x20), mload(_code))
jumpi(invalidJumpLabel, iszero(extcodesize(o_addr))) }
```

Parameters

_value, _code

Exploitation steps

- Obtain control over bytecode

Example payloads

- Malicious bytecode

Conditions

Access to memory storage.

Remediation

Use secure deserialization methods, such as using a trusted library or validating user input.

Secure example

```
function create(uint _value, bytes _code) internal returns (address o_addr)
{ assembly { o_addr := ecall(bytes32(0x12345678)) } }
```

File 7: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.643

Source lines: 1-61

► Finding 7.1: Insecure Deserialization (Critical)

Vulnerable code

```
players[tot] = Player(msg.sender, number);
```

Explanation

Untrusted data is deserialized without proper validation or sanitization.

Impact

Remote code execution and privilege escalation.

Entry point

/play(uint number) payable

Execution path

```
msg.value != 1 ether ? throw : players[tot] = Player(msg.sender, number);
```

Parameters

number

Exploitation steps

- Obtain arbitrary input for the 'number' parameter.

Example payloads

- A malicious 'number' value

Conditions

msg.sender != owner

Remediation

Use safe deserialization methods, validate object types, and implement integrity checks.

Secure example

```
players[tot] = Player(msg.sender, number); require(msg.sender == owner ||  
msg.value == 1 ether);
```

File 8: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.598

Source lines: 1-31

► Finding 8.1: Insecure Deserialization (Critical)

Vulnerable code

```
receiver.transfer(amount);
```

Explanation

Directly deserializes `amount` from untrusted data without validation, allowing for potential code execution.

Impact

Remote code execution

Entry point

/sendTo

Execution path

```
Sender -> Contract -> Receiver (transfer)\nReceiver -> Execution Context (execute)
```

Parameters

amount

Exploitation steps

- Send malicious amount to the contract

Example payloads

- `0x1234567890123456789012345678901234567890`

Conditions

The sender has control over the contract's state.

Remediation

Use safe deserialization methods or validate object types before passing user input.

Secure example

```
uint amount = msg.sender.value; receiver.transfer(amount);
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)